Operating Systems: Worksheet 1

This project shows the basics of 32-bit x86 assembly programming.

I've written several programs that work directly with CPU registers and memory, interfaced them with C code, and automated everything with a Makefile.

—----------

How to Run the Code

I've set up a Makefile to make building and running the code super easy.

1. **Clean Start:** First, clear out any old files to ensure a fresh build.
   make clean

2. **Build Everything:** Compile all the programs at once.
   make

3. **Run the Programs:** You can run each task individually from the terminal:
   ./task1 # Simple addition
   ./task1.2 # Interactive addition
   ./task2.1 # Name and welcome loop
   ./task2.2 # Sum numbers 1-100
   ./task2.3 # Sum a specific range

Task Breakdown

Here is what I built for each task, how it works, and proof that it runs successfully.

Task 1:

The goal here was to get comfortable with the basics: moving data into registers, doing math, and printing the results.

Task 1.1: Simple Addition

This program takes two hard-coded numbers (15 and 6), adds them together, and prints the result.

How it works:
It loads the numbers from memory into the EAX register, adds them up, and then calls print_int to show the answer.

```
; --- Code Snippet from task1.asm ---
mov     eax, [num1]   ; Load 15 into EAX
add     eax, [num2]   ; Add 6 to EAX (15 + 6 = 21)
call    print_string        ; Print the result (21)
```

driver.c | task1.asm | COMMIT_EDITMSG | README.md | Makefile | task1.2.o

src > task1.asm

```
%include "asm_io.inc"
;notes
segment .data
    ;two integers to add
    num1    dd  15          ; First number (15)
    num2    dd  6          ; Second number (6)
    result  dd  0           ; Storage for result
    msg1    db  "The sum is: ", 0

segment .bss
    ; No uninitialized data needed

segment .text
    global asm_main

asm_main:
    enter   0,0             ; Setup stack frame
    pusha                   ; Save all registers

    ; Print message
    mov     eax, msg1
    call    print_string

    ; Load first number into EAX storage
    mov     eax, [num1]     ; EAX = num1 (15)

    ; Add second number to EAX storage
    add     eax, [num2]     ; EAX = EAX + num2 (15 + 6 = 21)

```

Output:

```
immanuel2.mohammed@live.uwe.ac.uk@csctcloud:~/os/OS-WKSHEET-1$ ./task1
The sum is: 21
```

Task 1.2: Interactive Calculator

This version is more dynamic; it prompts the user for the numbers, adds them, and prints a formatted sentence.

How it works:
I used the read_int function to get input from the keyboard and stored those values in reserved memory slots.

```
; --- Code Snippet from task1.2.asm ---
call    read_int        ; Wait for user input
mov     [integer1], eax   ; Save the first number
...
add     eax, [integer1]   ; Add the first number to the second
call    print_int       ; Print the final sum
```

driver.c | task1.asm | *task1.2.asm* | COMMIT_EDITMSG | README.md | Makefile

src > task1.2.asm

```
13    segment .bss
16        result   resd 1
17
18    segment .text
19        global asm_main
20    asm_main:
21        pusha
22
23        ; Prompt for and read the first integer
24        mov     eax, msg1
25        call    print_string
26        call    read_int
27        mov     [integer1], eax
28
29        ; Prompt for and read the second integer
30        mov     eax, msg1
31        call    print_string
32        call    read_int
33        mov     [integer2], eax
34
35        ; --- Calculate the sum ---
36        mov     eax, [integer1]
37        add     eax, [integer2]
38        mov     [result], eax
39
40        ; --- Print the full output string ---
41        ; "The sum of "
42        mov     eax, msg2
43        call    print_string
```

Output:

```
immanuel2.mohammed@live.uwe.ac.uk@csctcloud:~/os/OS-WKSHEET-1$ ./task1.2
Enter a number: 2
Enter a number: 3
The sum of 2 and 3 is: 5
```

Task 2: Loops and Logic

This task was about controlling the flow of the program using loops and if/else logic (conditional jumps).

Task 2.1: The Welcome Loop

This program asks for your name and a number (between 51-99). It then says "Welcome" to you that many times.

How it works:

- **Reading a String:** The provided library didn't have a "read string" function, so I had to write my own loop. It reads one character at a time until you hit Enter.
- **Validation:** It checks if your number is valid (51-99) using cmp (compare). If it's too high or too low, it jumps to an error message.
- **The Loop:** It uses the loop instruction, which uses the ECX register as a countdown timer.

```
; --- Code Snippet from task2.1.asm ---
.read_name_loop:        ; My custom loop to read a name
    call    read_char
    cmp     eax, 10     ; Is it the Enter key?
    je      .done_reading
    mov     [edi], al   ; Save the character
    inc     edi         ; Move to next spot in memory
    jmp     .read_name_loop
```

Output:

```
immanuel2.mohammed@live.uwe.ac.uk@csctcloud:~/os/OS-WKSHEET-1$ ./task2.1
Enter your name: george
Enter a number (51-99): 51
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
Welcome, george
```

Use case error message when a number outside the range is inputted.

```
immanuel2.mohammed@live.uwe.ac.uk@csctcloud:~/os/OS-WKSHEET-1$ ./task2.1
Enter your name: ryan
Enter a number (51-99): 1
Error: The number is outside the valid range.
immanuel2.mohammed@live.uwe.ac.uk@csctcloud:~/os/OS-WKSHEET-1$
```

Task 2.2: Summing 1 to 100

This program creates an array (a list) of 100 numbers and then calculates the total sum.

How it works:
It uses two loops. The first one fills the array with numbers from 1 - 100. The second loop walks through the array, adding each number to a running total in the EBX register.

```
; --- Code Snippet from task2.2.asm ---
.sum_loop:
    mov     eax, [esi]    ; Get number from array
    add     ebx, eax      ; Add it to total sum
    add     esi, 4        ; Move to next number (4 bytes away)
    loop    .sum_loop
```

Output:

```
immanuel2.mohammed@live.uwe.ac.uk@csctcloud:~/os/OS-WKSHEET-1$ ./task2.2
Initializing array with values 1-100...
Sum of array: 5050
```

Task 2.3: Summing a Range

This extends the previous program. You get to choose the start and end of the sum (e.g., "sum numbers from 5 to 10").

How it works:
The tricky part here was calculating the correct starting memory address. I used the LEA (Load Effective Address) instruction to figure out exactly where in memory the summation should begin based on your input.

Output:

```
immanuel2.mohammed@live.uwe.ac.uk@csctcloud:~/os/OS-WKSHEET-1$ ./task2.3
Array initialized with values 1-100
Enter start index (0-99): 1
Enter end index (0-99): 30
Sum of elements from index 1 to 30 is: 495
```

Task 3: Automating with Make

To avoid typing long compilation commands every time, I created a Makefile. This script automates the entire build process.

Makefile

```
.PHONY: all clean

# Default rule
all: task1 task1.2 task2.1 task2.2 task2.3

# --- Rules to Link Final Executables ---
task1: driver.o asm_io.o task1.o
    gcc -m32 -no-pie driver.o task1.o asm_io.o -o task1

task1.2: driver.o asm_io.o task1.2.o
    gcc -m32 -no-pie driver.o task1.2.o asm_io.o -o task1.2

task2.1: driver.o asm_io.o task2.1.o
    gcc -m32 -no-pie driver.o task2.1.o asm_io.o -o task2.1

task2.2: driver.o asm_io.o task2.2.o
    gcc -m32 -no-pie driver.o task2.2.o asm_io.o -o task2.2

task2.3: driver.o asm_io.o task2.3.o
    gcc -m32 -no-pie driver.o task2.3.o asm_io.o -o task2.3

# --- Rules to Compile Object Files ---

# Shared object files
driver.o: src/driver.c
```

Makefile

```
20  task2.2: driver.o asm_io.o task2.2.o
22
23  task2.3: driver.o asm_io.o task2.3.o
24      gcc -m32 -no-pie driver.o task2.3.o asm_io.o -o task2.3
25
26  # --- Rules to Compile Object Files ---
27
28  # Shared object files
29  driver.o: src/driver.c
30      gcc -m32 -c src/driver.c -o driver.o
31
32  asm_io.o: src/asm_io.asm src/asm_io.inc
33      nasm -f elf -Isrc/ src/asm_io.asm -o asm_io.o
34
35  # --- SMART PATTERN RULE ---
36  # This rule says: "To build ANY .o file (like task1.o),
37  # look for the matching .asm file inside src/ (like src/task1.asm)."
38  # It automatically adds the src/ prefix and the -Isrc/ flag.
39  %.o: src/%.asm src/asm_io.inc
40      nasm -f elf -Isrc/ $< -o $@
41
42  # --- Cleanup Rule ---
43  # UPDATED: Now cleans .o files in both root and src/ directories
44  clean:
45      rm -f *.o src/*.o task1 task1.2 task2.1 task2.2 task2.3
```

Challenges & Fixes:

I ran into a few bugs while setting this up, which was a great learning experience:

1. **The "Missing File" Error:** The assembler couldn't find asm_io.inc.
   - *Fix:* I had to tell the assembler (nasm) to look inside the src/ folder by adding the -Isrc/ flag.
2. **The "Underscore" Mix-up:** My files were named task2_2.asm, but the Makefile expected task2.2.asm.
   - *Fix:* I renamed the files to be consistent with dots (e.g., task2.2.asm).
3. **The "Clean" Issue:** make clean wasn't deleting the old files inside the src/ folder.
   - *Fix:* I updated the clean command to remove src/*.o files too.